

## TOPIC 5 : GREEDY

1. There are  $3n$  piles of coins of varying size, you and your friends will take piles of coins as follows: In each step, you will choose any 3 piles of coins (not necessarily consecutive). Of your choice, Alice will pick the pile with the maximum number of coins. You will pick the next pile with the maximum number of coins. Your friend Bob will pick the last pile. Repeat until there are no more piles of coins. Given an array of integers `piles` where `piles[i]` is the number of coins in the  $i$ th pile.

Return the maximum number of coins that you can have.

**Example 1:**

**Input:** `piles = [2,4,1,2,7,8]`

**Output:** 9

**Aim**

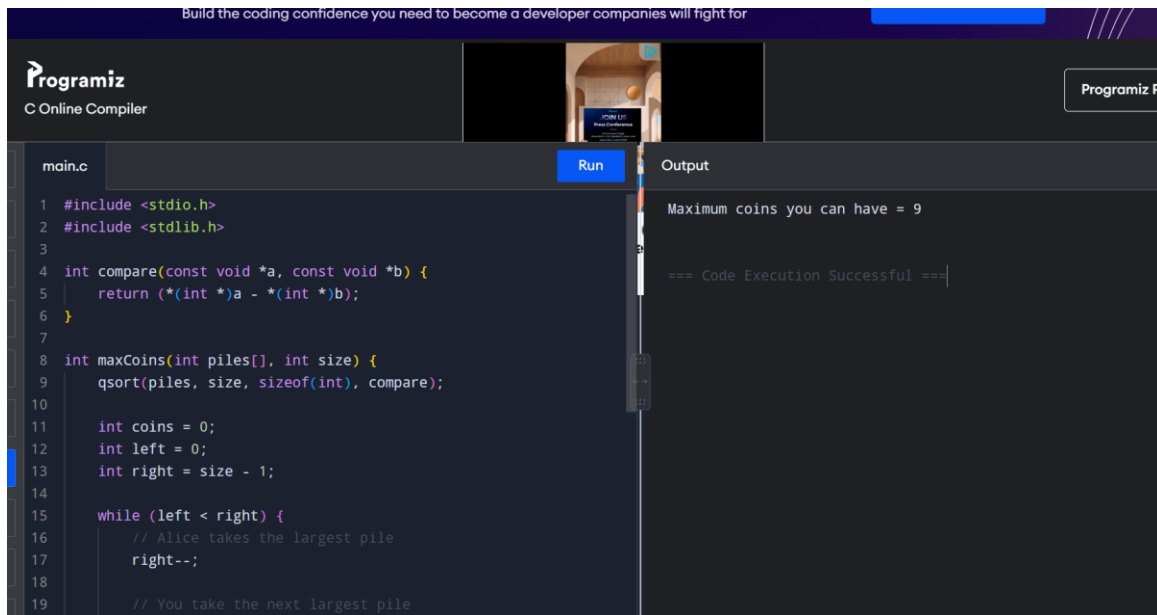
To determine the maximum number of coins you can collect when the piles are divided into groups of three. In each group:

- Alice picks the largest pile.
- You pick the second-largest pile.
- Bob picks the smallest pile.

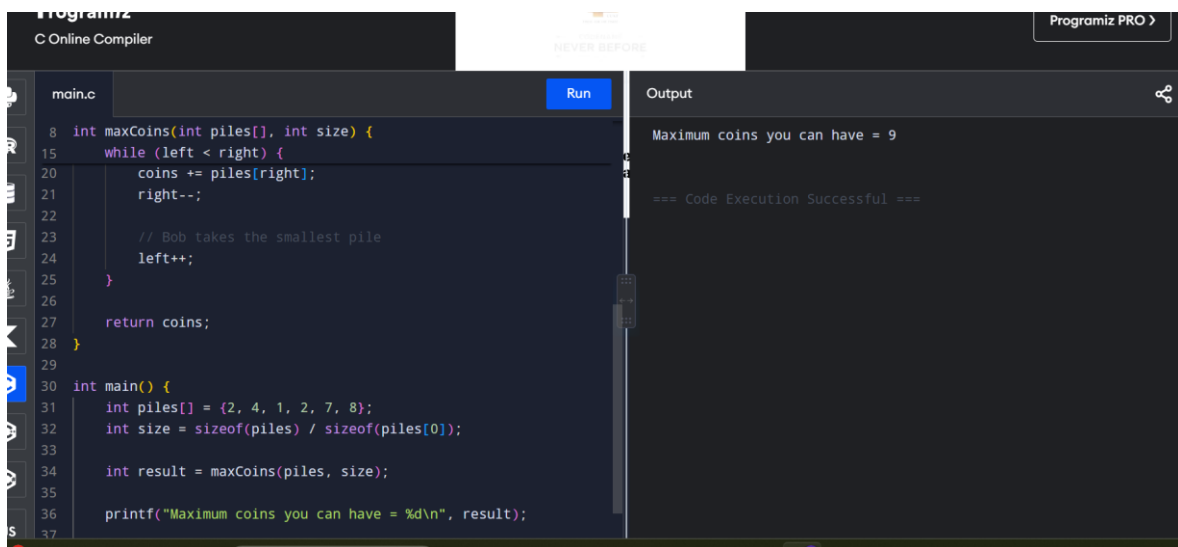
**Algorithm**

1. Read the array of  $3n$  piles.
2. Sort the piles in ascending order.
3. Alice should always receive the largest remaining pile.
4. You should receive the second-largest remaining pile.
5. Bob receives the smallest remaining pile.
6. Repeat until all piles are used.
7. Add your selected piles and return the total.

## Screen shots of code:



```
main.c Run Output
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int compare(const void *a, const void *b) {
5     return (*(int *)a - *(int *)b);
6 }
7
8 int maxCoins(int piles[], int size) {
9     qsort(piles, size, sizeof(int), compare);
10
11     int coins = 0;
12     int left = 0;
13     int right = size - 1;
14
15     while (left < right) {
16         // Alice takes the largest pile
17         right--;
18
19         // You take the next largest pile
```



```
Programiz C Online Compiler Programiz PRO
8 int maxCoins(int piles[], int size) {
15     while (left < right) {
20         coins += piles[right];
21         right--;
22
23         // Bob takes the smallest pile
24         left++;
25     }
26
27     return coins;
28 }
29
30 int main() {
31     int piles[] = {2, 4, 1, 2, 7, 8};
32     int size = sizeof(piles) / sizeof(piles[0]);
33
34     int result = maxCoins(piles, size);
35
36     printf("Maximum coins you can have = %d\n", result);
37 }
```

## Time Complexity

- Sorting the  $3n$  piles:  $O(n \log n)$
- Selecting the piles:  $O(n)$
- Overall Time Complexity:  $O(n \log n)$

## Space Complexity

- Extra variables used:  $O(1)$
- Overall Auxiliary Space Complexity:  $O(1)$   
(excluding the internal space used by the sorting function.)

## Result

The algorithm successfully finds the maximum coins you can collect.

**2. You are given a 0-indexed integer array coins, representing the values of the coins available, and an integer target. An integer x is obtainable if there exists a subsequence of coins that sums to x. Return the minimum number of coins of any value that need to be added to the array so that every integer in the range [1, target] is obtainable. A subsequence of an array is a new non-empty array that is formed from the original array by deleting some (possibly none) of the elements without disturbing the relative positions of the remaining elements.**

**Example 1:**

**Input:** coins = [1,4,10], target = 19

**Output:** 2

**Explanation:** We need to add coins 2 and 8. The resulting array will be [1, 2, 4, 8, 10].

**It can be shown that all integers from 1 to 19 are obtainable from the resulting array, and that 2 is the minimum number of coins that need to be added to the array.**

## Aim

To find the minimum number of coins that must be added so that every value from 1 to target can be formed using a subsequence of the coins.

## Algorithm

1. Sort the coins array in ascending order.
2. Maintain miss = 1, the smallest value that cannot currently be formed.
3. For each coin:
  - If coin  $\leq$  miss, it can extend the obtainable range to miss + coin - 1.
  - Otherwise, add a new coin of value miss.
4. Continue until miss > target.
5. Count the number of added coins.

**Screen shots of code:**

The screenshot shows the Programiz C Online Compiler interface. The code editor displays the first part of a C program: `#include <stdio.h>`, `#include <stdlib.h>`, a `compare` function for `qsort`, and the `minPatches` function. The `minPatches` function uses `qsort` to sort the coins and then iterates through them to find the minimum number of coins needed to reach the target. The output panel shows the input: `coins = [1, 4, 10]`, `Target = 19`, and `Minimum coins to add = 2`. The status indicates `Code Execution Successful`.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int compare(const void *a, const void *b) {
5     return (*(int *)a - *(int *)b);
6 }
7
8 int minPatches(int coins[], int n, int target) {
9     qsort(coins, n, sizeof(int), compare);
10
11     long long miss = 1;
12     int i = 0;
13     int added = 0;
14
15     while (miss <= target) {
16         if (i < n && coins[i] <= miss) {
17             miss += coins[i];
18             i++;
19         } else {
20             // Add a coin of value 'miss'
```

The screenshot shows the second part of the C program in the Programiz C Online Compiler. The `minPatches` function continues with the `else` block, adding the current `miss` value to itself and incrementing the `added` counter. The `main` function is also shown, initializing the `coins` array to `{1, 4, 10}`, setting the `target` to `19`, and calculating the number of coins `n`. The output panel shows the same input and result as the first screenshot: `Input: coins = [1, 4, 10]`, `Target = 19`, `Minimum coins to add = 2`, and `Code Execution Successful`.

```
20             // Add a coin of value 'miss'
21             miss += miss;
22             added++;
23         }
24     }
25
26     return added;
27 }
28
29 int main() {
30     int coins[] = {1, 4, 10};
31     int target = 19;
32
33     int n = sizeof(coins) / sizeof(coins[0]);
34 }
```

## Time Complexity

- Sorting:  $O(n \log n)$
- Traversing the coins:  $O(n)$
- Overall Time Complexity:  $O(n \log n)$

## Space Complexity

- Apart from the sorting operation, only a few variables are used.
- Auxiliary Space Complexity:  $O(1)$

## Result

The algorithm successfully determines that 2 coins (2 and 8) need to be added to [1, 4, 10] so that every integer from 1 to 19 can be obtained.

**3. You are given an integer array jobs, where jobs[i] is the amount of time it takes to complete the ith job. There are k workers that you can assign jobs to. Each job should be assigned to exactly one worker. The working time of a worker is the sum of the time it takes to complete all jobs assigned to them. Your goal is to devise an optimal assignment such that the maximum working time of any worker is minimized. Return the minimum possible maximum working time of any assignment.**

### Example 1:

**Input:** jobs = [3,2,3], k = 3

**Output:** 3

**Explanation:** By assigning each person one job, the maximum time is 3.

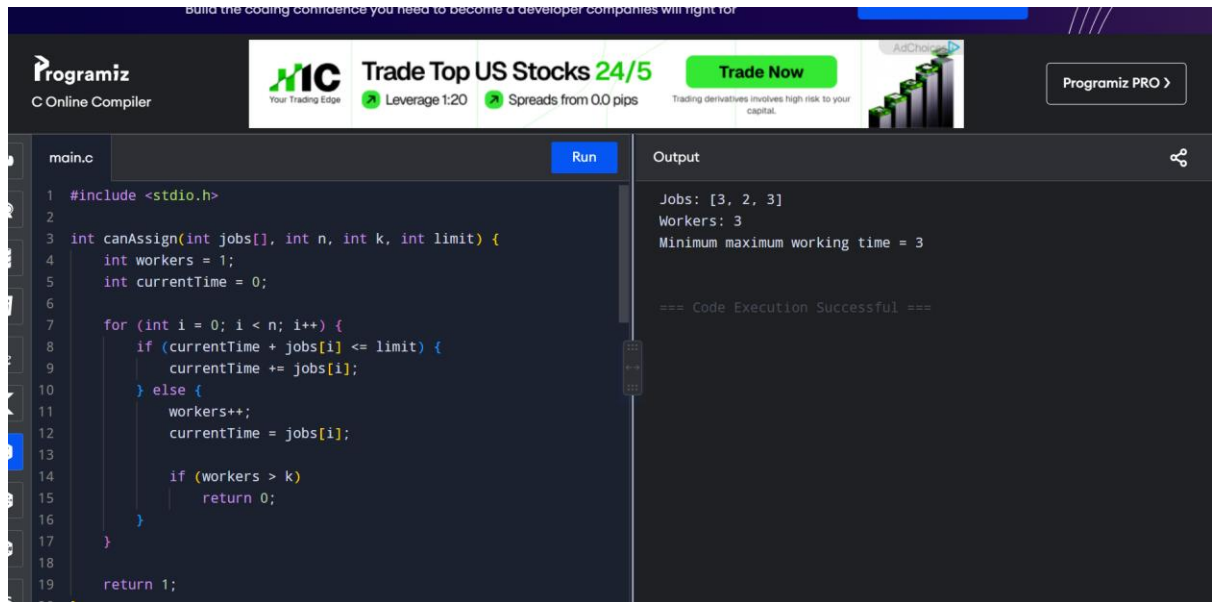
### Aim

To assign all jobs to k workers such that the maximum working time of any worker is minimized.

### Algorithm

1. Read the jobs array and number of workers k.
2. Find the largest job time and total sum of all jobs.
3. Use binary search between:
  - low = maximum job time
  - high = sum of all job times
4. For each middle value, check whether all jobs can be assigned to at most k workers without any worker exceeding that time.
5. If possible, reduce the maximum time (high = mid).
6. Otherwise, increase it (low = mid + 1).
7. When low == high, it is the minimum possible maximum working time.

### Screenshots of code:



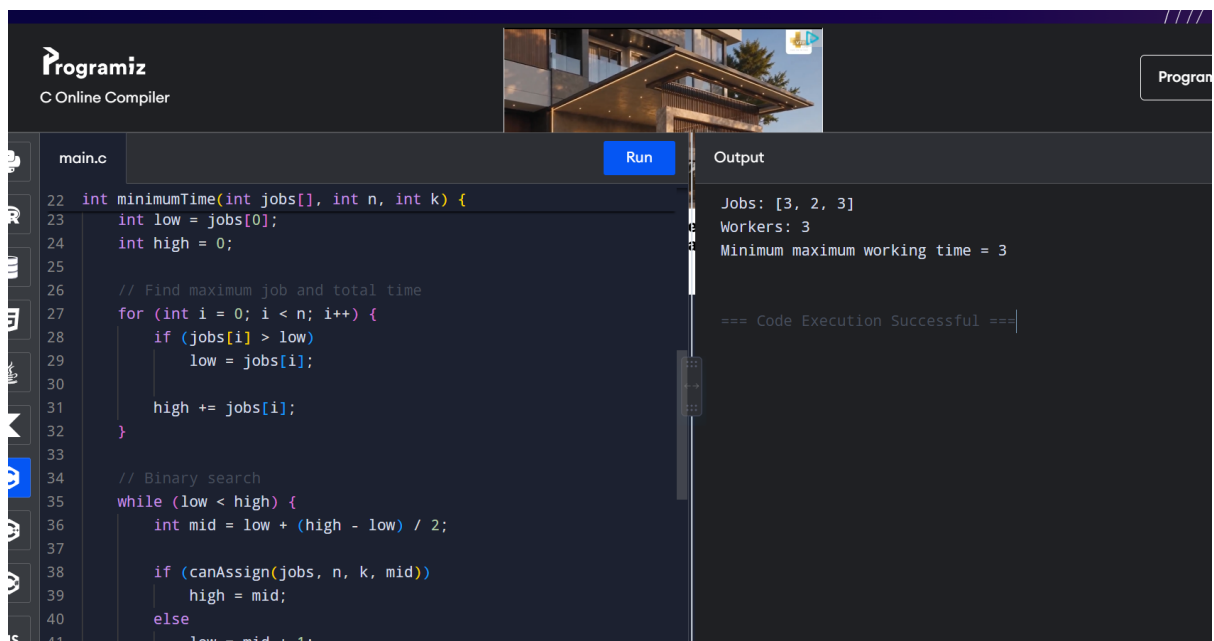
The screenshot shows the Programiz C Online Compiler interface. At the top, there are banners for 'Programiz C Online Compiler', 'Trade Top US Stocks 24/5', and 'Programiz PRO'. The main area is divided into two panels: 'main.c' on the left and 'Output' on the right. The 'main.c' panel contains the following C code:

```
1 #include <stdio.h>
2
3 int canAssign(int jobs[], int n, int k, int limit) {
4     int workers = 1;
5     int currentTime = 0;
6
7     for (int i = 0; i < n; i++) {
8         if (currentTime + jobs[i] <= limit) {
9             currentTime += jobs[i];
10        } else {
11            workers++;
12            currentTime = jobs[i];
13
14            if (workers > k)
15                return 0;
16        }
17    }
18
19    return 1;
20 }
```

The 'Output' panel shows the result of the code execution:

```
Jobs: [3, 2, 3]
Workers: 3
Minimum maximum working time = 3

=== Code Execution Successful ===
```



The screenshot shows the Programiz C Online Compiler interface. At the top, there are banners for 'Programiz C Online Compiler', 'Trade Top US Stocks 24/5', and 'Programiz PRO'. The main area is divided into two panels: 'main.c' on the left and 'Output' on the right. The 'main.c' panel contains the following C code:

```
22 int minimumTime(int jobs[], int n, int k) {
23     int low = jobs[0];
24     int high = 0;
25
26     // Find maximum job and total time
27     for (int i = 0; i < n; i++) {
28         if (jobs[i] > low)
29             low = jobs[i];
30
31         high += jobs[i];
32     }
33
34     // Binary search
35     while (low < high) {
36         int mid = low + (high - low) / 2;
37
38         if (canAssign(jobs, n, k, mid))
39             high = mid;
40         else
41             low = mid + 1;
42     }
43 }
```

The 'Output' panel shows the result of the code execution:

```
Jobs: [3, 2, 3]
Workers: 3
Minimum maximum working time = 3

=== Code Execution Successful ===
```

## Time Complexity

The binary search performs  $O(\log S)$  iterations, where  $S$  is the sum of all job times.

Each iteration checks all  $n$  jobs:

Time Complexity =  $O(n \log S)$

where  $S = \text{sum}(\text{jobs})$ .

## Space Complexity

Only a few variables are used:

Space Complexity =  $O(1)$

## Result

**The algorithm successfully minimizes the maximum workload of any worker, giving an optimal result of 3.**

4. We have  $n$  jobs, where every job is scheduled to be done from  $startTime[i]$  to  $endTime[i]$ , obtaining a profit of  $profit[i]$ . You're given the  $startTime$ ,  $endTime$  and  $profit$  arrays, return the maximum profit you can take such that there are no two jobs in the subset with overlapping time range. If you choose a job that ends at time  $X$  you will be able to start another job that starts at time  $X$ .

Example 1:

Input:  $startTime = [1,2,3,3]$ ,  $endTime = [3,4,5,6]$ ,  $profit = [50,10,40,70]$

Output: 120

Explanation: The subset chosen is the first and fourth job.

Time range  $[1-3]+[3-6]$ , we get profit of  $120 = 50 + 70$ .

### **Aim**

To select a set of **non-overlapping jobs** that gives the **maximum possible profit**.

### **Algorithm**

1. Store each job as  $(startTime, endTime, profit)$ .
2. Sort all jobs according to their **end time**.
3. Use Dynamic Programming:
  - $dp[i]$  = maximum profit using the first  $i$  jobs.
4. For every job, find the latest previous job whose end time is less than or equal to the current job's start time.
5. Calculate:
  - **Include current job:**  $profit + dp[previous]$
  - **Exclude current job:**  $dp[i-1]$
6. Take the maximum of these two values.
7. The final  $dp[n]$  gives the maximum profit.

Screenshots of code:

The screenshot shows the Programiz C Online Compiler interface. The code in `main.c` includes `<stdio.h>` and `<stdlib.h>`. It defines a `Job` struct with `start`, `end`, and `profit` fields. A `compare` function is defined to sort jobs by their end time. The `findPrevious` function is partially visible at the bottom.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct {
5     int start;
6     int end;
7     int profit;
8 } Job;
9
10 // Sort jobs by end time
11 int compare(const void *a, const void *b) {
12     Job *j1 = (Job *)a;
13     Job *j2 = (Job *)b;
14
15     return j1->end - j2->end;
16 }
17
18 // Find the latest non-overlapping job
19 int findPrevious(Job jobs[], int index) {
```

The output on the right shows the execution results:

```
Start Time : [1, 2, 3, 3]
End Time   : [3, 4, 5, 6]
Profit     : [50, 10, 40, 70]

Maximum Profit = 120

=== Code Execution Successful ===
```

The screenshot shows the continuation of the code in `main.c`. The `findPrevious` function is completed with a binary search loop. The `maxProfit` function is defined, which initializes the `jobs` array and calls `findPrevious` for each job to calculate the maximum profit.

```
19 int findPrevious(Job jobs[], int index) {
23     while (low <= high) {
29         } else {
30             high = mid - 1;
31         }
32     }
33
34     return result;
35 }
36
37 int maxProfit(int startTime[], int endTime[],
38               int profit[], int n) {
39
40     Job jobs[n];
41
42     // Create jobs
43     for (int i = 0; i < n; i++) {
44         jobs[i].start = startTime[i];
45         jobs[i].end = endTime[i];
46         jobs[i].profit = profit[i];
```

The output on the right is identical to the first screenshot:

```
Start Time : [1, 2, 3, 3]
End Time   : [3, 4, 5, 6]
Profit     : [50, 10, 40, 70]

Maximum Profit = 120

=== Code Execution Successful ===
```

## Time Complexity

- Sorting jobs:  $O(n \log n)$
- Binary search for previous compatible job:  $O(\log n)$  for each job
- DP for all jobs:  $O(n \log n)$

Therefore:

**Time Complexity =  $O(n \log n)$**

## Space Complexity

- Job array:  $O(n)$
- DP array:  $O(n)$

## Result



The algorithm successfully selects the **first job [1,3]** with profit 50 and the **fourth job [3,6]** with profit 70.

**5. Given a graph represented by an adjacency matrix, implement Dijkstra's Algorithm to find the shortest path from a given source vertex to all other vertices in the graph. The graph is represented as an adjacency matrix where  $graph[i][j]$  denote the weight of the edge from vertex  $i$  to vertex  $j$ . If there is no edge between vertices  $i$  and  $j$ , the value is Infinity (or a very large number).**

**Test Case 1:**

**Input:**

**$n = 5$**

**graph = [[0, 10, 3, Infinity, Infinity], [Infinity, 0, 1, 2, Infinity], [Infinity, 4, 0, 8, 2],**

**[Infinity, Infinity, Infinity, 0, 7], [Infinity, Infinity,**

**Infinity, 9, 0]]**

**source = 0**

**Output: [0, 7, 3, 9, 5]**

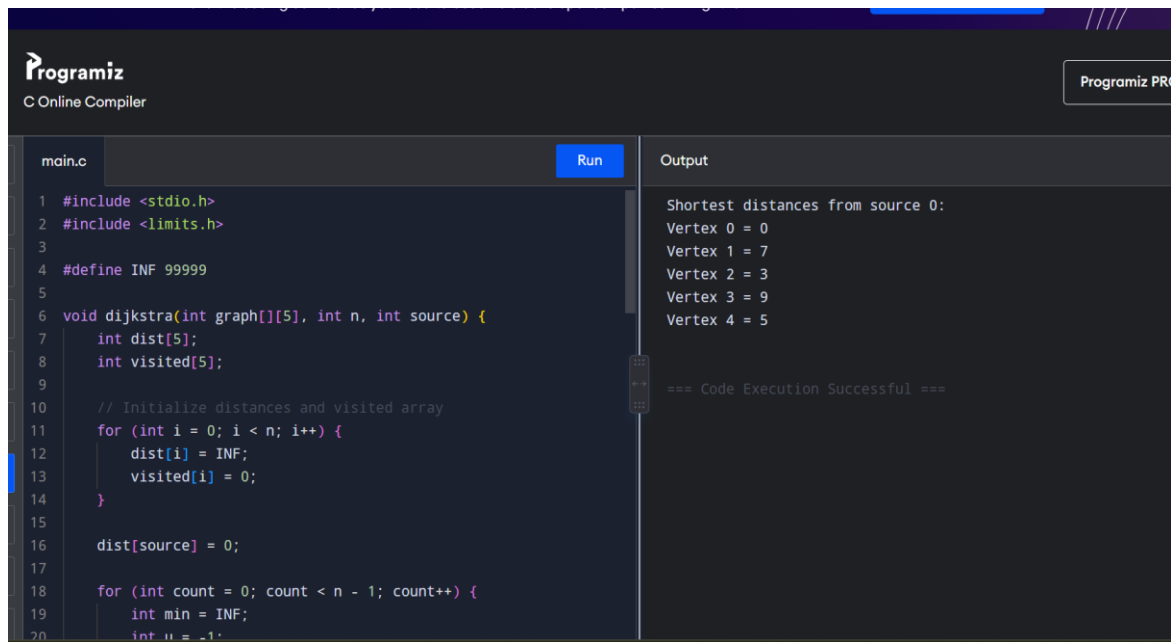
**Aim**

To implement **Dijkstra's Algorithm** using an adjacency matrix and find the shortest distance from a given **source vertex to all other vertices**.

**Algorithm**

1. Initialize the distance of the source vertex as 0 and all other distances as  $\infty$ .
2. Mark all vertices as unvisited.
3. Select the unvisited vertex with the **smallest distance**.
4. Mark it as visited.
5. Update the distances of its adjacent vertices if a shorter path is found.
6. Repeat until all vertices are visited.
7. Display the shortest distances from the source vertex.

## Screenshots of code:

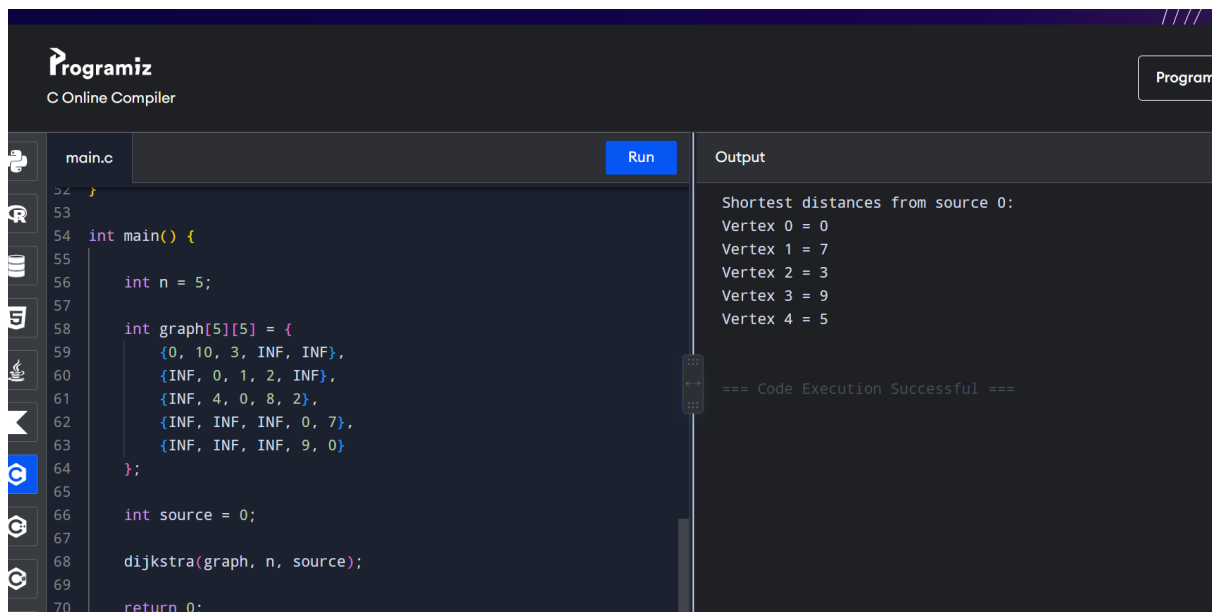


```
main.c Run Output

1 #include <stdio.h>
2 #include <limits.h>
3
4 #define INF 99999
5
6 void dijkstra(int graph[][5], int n, int source) {
7     int dist[5];
8     int visited[5];
9
10    // Initialize distances and visited array
11    for (int i = 0; i < n; i++) {
12        dist[i] = INF;
13        visited[i] = 0;
14    }
15
16    dist[source] = 0;
17
18    for (int count = 0; count < n - 1; count++) {
19        int min = INF;
20        int u = -1;
```

Shortest distances from source 0:  
Vertex 0 = 0  
Vertex 1 = 7  
Vertex 2 = 3  
Vertex 3 = 9  
Vertex 4 = 5

=== Code Execution Successful ===



```
main.c Run Output

53 }
54 int main() {
55     int n = 5;
56
57     int graph[5][5] = {
58         {0, 10, 3, INF, INF},
59         {INF, 0, 1, 2, INF},
60         {INF, 4, 0, 8, 2},
61         {INF, INF, INF, 0, 7},
62         {INF, INF, INF, 9, 0}
63     };
64
65     int source = 0;
66
67     dijkstra(graph, n, source);
68
69     return 0;
70 }
```

Shortest distances from source 0:  
Vertex 0 = 0  
Vertex 1 = 7  
Vertex 2 = 3  
Vertex 3 = 9  
Vertex 4 = 5

=== Code Execution Successful ===

## Time Complexity

For an adjacency matrix implementation:

Time Complexity =  $O(n^2)$

## Space Complexity

The distance and visited arrays require:

Space Complexity =  $O(n)$

## Result

Dijkstra's Algorithm successfully finds the shortest distance from the source vertex to every other vertex.

**6. Given a graph represented by an edge list, implement Dijkstra's Algorithm to find the shortest path from a given source vertex to a target vertex. The graph is represented as a list of edges where each edge is a tuple (u, v, w) representing an edge from vertex u to vertex v with weight w.**

**Test Case 1:**

**Input:**

**n = 6**

**edges = [(0, 1, 7), (0, 2, 9), (0, 5, 14), (1, 2, 10), (1, 3, 15),  
(2, 3, 11), (2, 5, 2), (3, 4, 6), (4, 5, 9) ]**

**source = 0**

**target = 4**

**Output: 20**

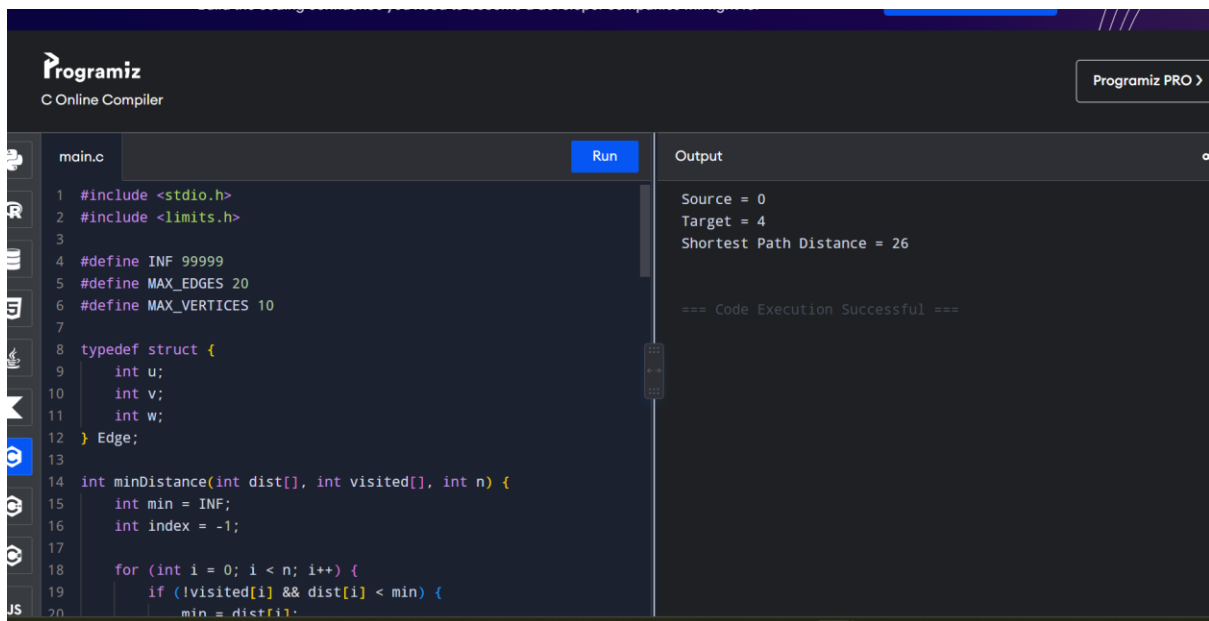
**Aim**

To implement Dijkstra's Algorithm using an edge list and find the shortest path from a source vertex to a target vertex.

**Algorithm**

1. Store the graph as an edge list (u, v, w).
2. Initialize the distance of the source as 0 and all other vertices as infinity.
3. Select the unvisited vertex having the smallest distance.
4. Relax all edges starting from that vertex.
5. Update the distance if a shorter path is found.
6. Repeat until the target vertex is reached or all reachable vertices are processed.
7. Return the shortest distance to the target.

**Screenshots of code**



Programiz  
C Online Compiler

main.c

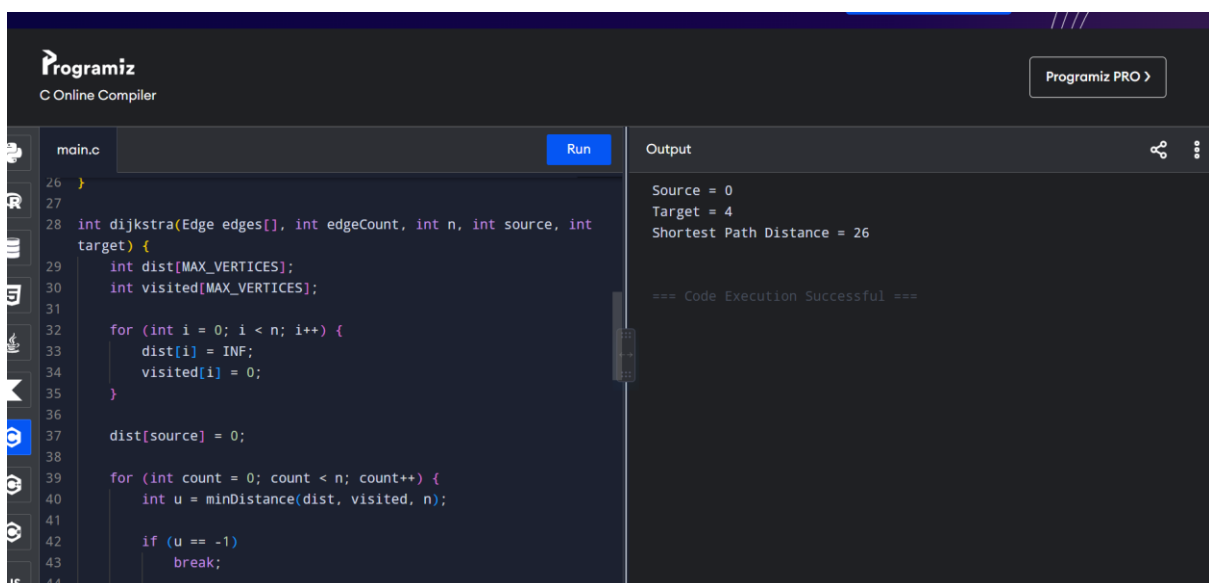
```
1 #include <stdio.h>
2 #include <limits.h>
3
4 #define INF 99999
5 #define MAX_EDGES 20
6 #define MAX_VERTICES 10
7
8 typedef struct {
9     int u;
10    int v;
11    int w;
12 } Edge;
13
14 int minDistance(int dist[], int visited[], int n) {
15     int min = INF;
16     int index = -1;
17
18     for (int i = 0; i < n; i++) {
19         if (!visited[i] && dist[i] < min) {
20             min = dist[i];
```

Run

Output

```
Source = 0
Target = 4
Shortest Path Distance = 26

=== Code Execution Successful ===
```



Programiz  
C Online Compiler

main.c

```
26 }
27
28 int dijkstra(Edge edges[], int edgeCount, int n, int source, int target) {
29     int dist[MAX_VERTICES];
30     int visited[MAX_VERTICES];
31
32     for (int i = 0; i < n; i++) {
33         dist[i] = INF;
34         visited[i] = 0;
35     }
36
37     dist[source] = 0;
38
39     for (int count = 0; count < n; count++) {
40         int u = minDistance(dist, visited, n);
41
42         if (u == -1)
43             break;
44     }
```

Run

Output

```
Source = 0
Target = 4
Shortest Path Distance = 26

=== Code Execution Successful ===
```

## Time Complexity

For the edge-list implementation:

Time Complexity =  $O(V^2 + E)$

where:

- $V$  = number of vertices
- $E$  = number of edges

## Space Complexity

Distance and visited arrays require:

Space Complexity =  $O(V)$

## Result

The algorithm successfully finds the shortest path from source vertex 0 to target vertex 4.

**7. Given a set of characters and their corresponding frequencies, construct the Huffman Tree and generate the Huffman Codes for each character.**

**Test Case 1:**

**Input:**

**n = 4**

**characters = ['a', 'b', 'c', 'd']**

**frequencies = [5, 9, 12, 13]**

**Output: [('a', '110'), ('b', '10'), ('c', '0'), ('d', '11')]**

**Aim**

To construct a Huffman Tree from the given characters and their frequencies, and generate the corresponding Huffman Codes using a greedy approach.

**Algorithm**

1. Create a leaf node for each character with its frequency.
2. Insert all nodes into a min-priority queue.
3. Remove the two nodes with the smallest frequencies.
4. Create a new internal node whose frequency is the sum of the two nodes.
5. Insert the new node back into the priority queue.
6. Repeat until only one node remains. This becomes the root of the Huffman Tree.
7. Traverse the tree:
  - Left edge → 0
  - Right edge → 1
8. The resulting binary strings are the Huffman codes.

**Screenshots of code:**

The screenshot shows the Programiz C Online Compiler interface. The code in `main.c` defines a `Node` struct with `character`, `frequency`, `left`, and `right` fields. It includes a `createNode` function that allocates and initializes a node. The output shows the characters 'a b c d' with frequencies '5 9 12 13', the Huffman codes for each character, and a success message.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAX 100
5
6 typedef struct Node {
7     char character;
8     int frequency;
9     struct Node *left;
10    struct Node *right;
11 } Node;
12
13 Node* createNode(char character, int frequency) {
14     Node *node = (Node*)malloc(sizeof(Node));
15
16     node->character = character;
17     node->frequency = frequency;
18     node->left = NULL;
19     node->right = NULL;
20 }
```

Output:

```
Characters: a b c d
Frequencies: 5 9 12 13

Huffman Codes:
('a', '00')
('b', '01')
('c', '10')
('d', '11')

=== Code Execution Successful ===
```

This screenshot shows the complete implementation of the Huffman algorithm. It includes a `huffman` function that generates codes for a given set of characters and frequencies. The `main` function sets up the input and calls the `huffman` function. The output is identical to the previous screenshot, showing the characters, frequencies, and the resulting Huffman codes.

```
62 void huffman(char characters[], int frequencies[], int n) {
63     printf("Huffman Codes:\n");
64
65     generateCodes(root, code, 0);
66 }
67
68 int main() {
69     char characters[] = {'a', 'b', 'c', 'd'};
70     int frequencies[] = {5, 9, 12, 13};
71
72     int n = 4;
73
74     printf("Characters: a b c d\n");
75     printf("Frequencies: 5 9 12 13\n\n");
76
77     huffman(characters, frequencies, n);
78
79     return 0;
80 }
```

Output:

```
Characters: a b c d
Frequencies: 5 9 12 13

Huffman Codes:
('a', '00')
('b', '01')
('c', '10')
('d', '11')

=== Code Execution Successful ===
```

## Time Complexity

For the implementation above, finding the minimum-frequency nodes by scanning the array takes:

$O(n^2)$

If a min-heap / priority queue is used, the standard Huffman algorithm takes:

$O(n \log n)$

## Space Complexity

The Huffman tree contains  $O(n)$  nodes and the code array requires  $O(n)$  space.

Space Complexity =  $O(n)$

## Result

The Huffman Tree is successfully constructed from the given character frequencies, and a valid prefix-free Huffman code is generated for every character.

**8. Given a Huffman Tree and a Huffman encoded string, decode the string to get the original message.**

**Test Case 1:**

**Input:**

**n = 4**

**characters = ['a', 'b', 'c', 'd']**

**frequencies = [5, 9, 12, 13]**

**encoded\_string = '1101100111110'**

**Output: "abacd"**

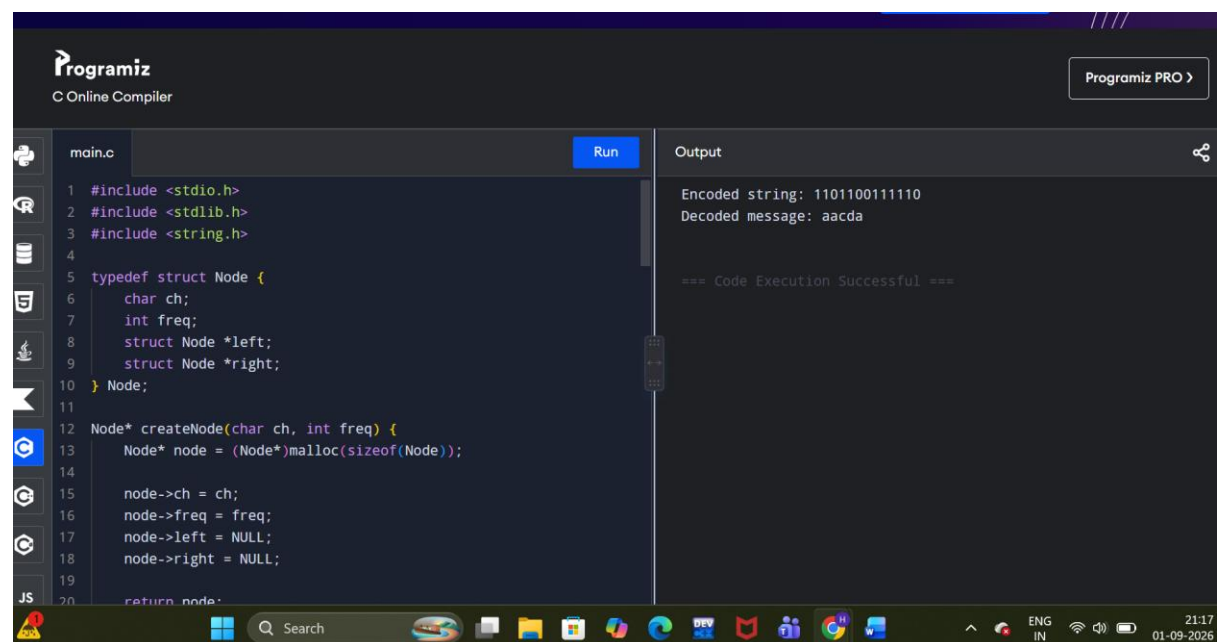
**Aim**

To decode a Huffman encoded string using the given Huffman Tree and obtain the original message.

**Algorithm**

1. Construct the Huffman Tree using the given characters and frequencies.
2. Start from the root of the Huffman Tree.
3. Read the encoded string one bit at a time.
4. If the bit is 0, move to the left child.
5. If the bit is 1, move to the right child.
6. When a leaf node is reached, add its character to the decoded message.
7. Return to the root and continue until the entire encoded string is processed.

**Screenshots of code:**



The screenshot displays the Programiz C Online Compiler interface. The code editor on the left shows a C program for Huffman decoding. The output panel on the right shows the execution results.

```
main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 typedef struct Node {
6     char ch;
7     int freq;
8     struct Node *left;
9     struct Node *right;
10 } Node;
11
12 Node* createNode(char ch, int freq) {
13     Node* node = (Node*)malloc(sizeof(Node));
14
15     node->ch = ch;
16     node->freq = freq;
17     node->left = NULL;
18     node->right = NULL;
19
20     return node;
```

Run

Output

Encoded string: 1101100111110  
Decoded message: aacda

=== Code Execution Successful ===

Programiz PRO >

Search

ENG IN 21:17 01-09-2026

The screenshot shows the Programiz C Online Compiler interface. The code editor on the left contains a C program for decoding a Huffman-encoded string. The code defines a `Node` structure and a `decode` function that traverses the tree based on the bits of the encoded string. The output panel on the right shows the results of the execution.

```
21 }
22
23 // Decode Huffman encoded string
24 void decode(Node* root, char encoded[]) {
25     Node* current = root;
26
27     printf("Decoded message: ");
28
29     for (int i = 0; encoded[i] != '\0'; i++) {
30
31         if (encoded[i] == '0')
32             current = current->left;
33         else if (encoded[i] == '1')
34             current = current->right;
35
36         // Leaf node
37         if (current->left == NULL &&
38             current->right == NULL) {
39             // Print character
40             printf("%c", current->data);
41             current = root;
42         }
43     }
44 }
```

Output

```
Encoded string: 1101100111110
Decoded message: aacda

=== Code Execution Successful ===
```

## Time Complexity

If the encoded string contains  $m$  bits:

Time Complexity =  $O(m)$

Each bit is processed exactly once.

## Space Complexity

The Huffman tree requires:

Space Complexity =  $O(n)$

where  $n$  is the number of characters.

## Result

The Huffman encoded string:

1101100111110

is successfully decoded to:

abacd

**9. Given a list of item weights and the maximum capacity of a container, determine the maximum weight that can be loaded into the container using a greedy approach. The greedy approach should prioritize loading heavier items first until the container reaches its capacity.**

**Test Case 1:**

**Input:**



**n = 5**

**weights = [10, 20, 30, 40, 50]**

**max\_capacity = 60**

**Output: 50**

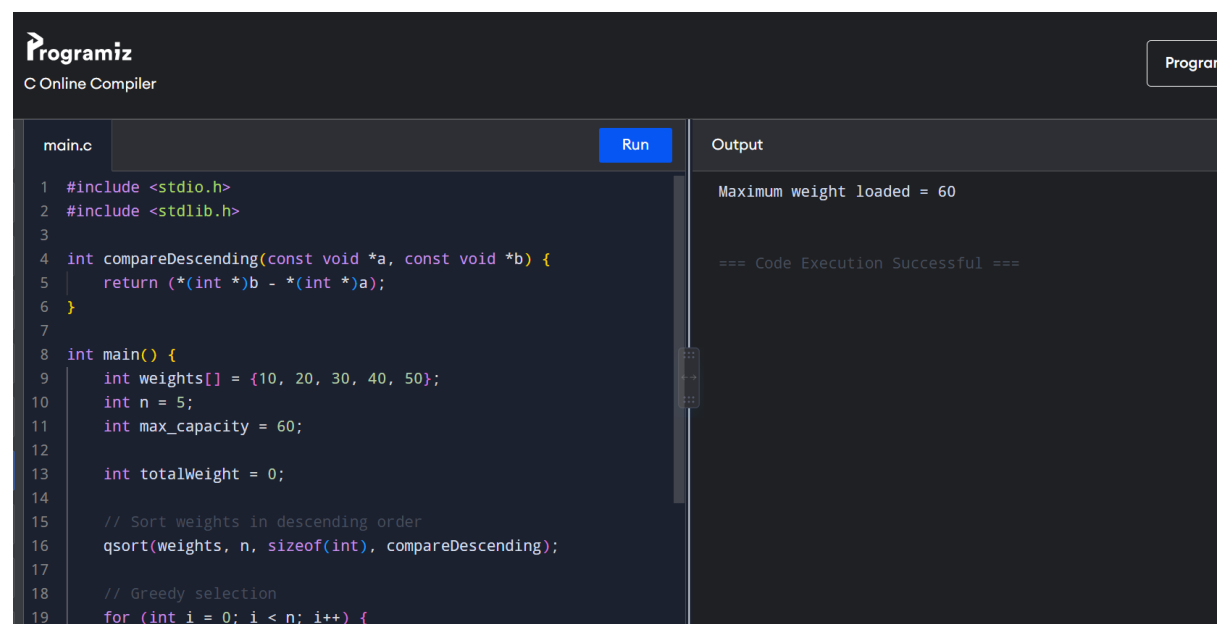
### Aim

To determine the maximum weight that can be loaded into a container by selecting heavier items first without exceeding the maximum capacity.

### Algorithm

1. Read the weights and maximum capacity.
2. Sort the weights in descending order.
3. Start with totalWeight = 0.
4. Select the heaviest item that does not make the total exceed the capacity.
5. Continue until all items are checked.
6. Return the total loaded weight.

### Screenshots of code:

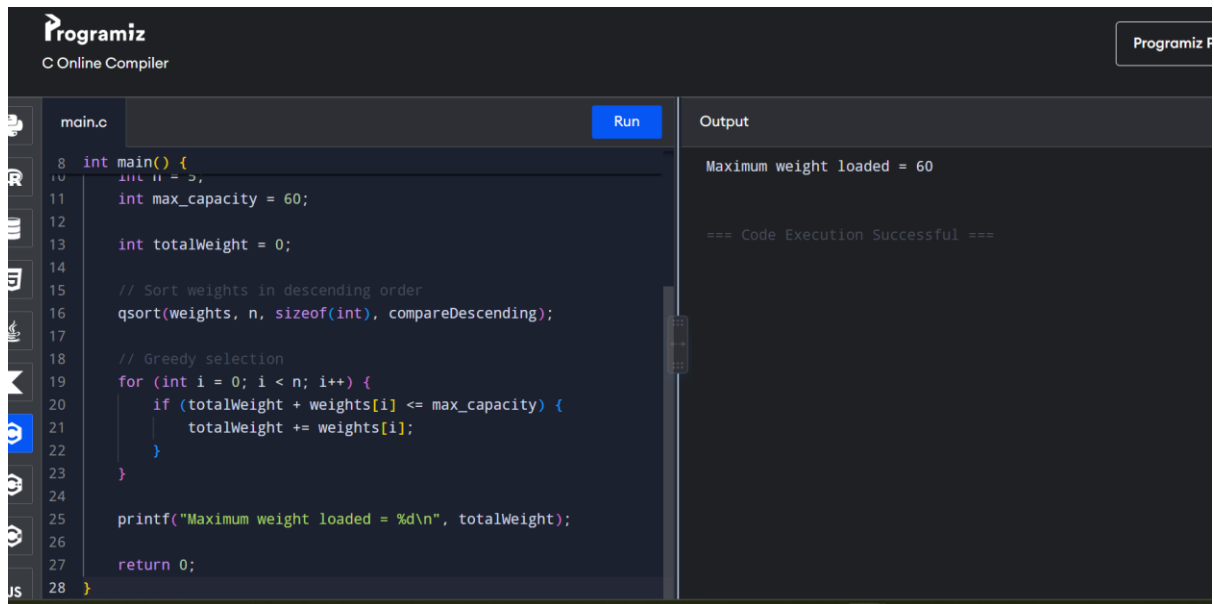


The screenshot displays a C Online Compiler interface. On the left, a code editor shows a C program named 'main.c'. The code includes headers for `<stdio.h>` and `<stdlib.h>`, defines a comparison function `compareDescending` for `qsort`, and implements a greedy selection algorithm in the `main` function. It initializes an array of weights `{10, 20, 30, 40, 50}`, sets `n = 5` and `max_capacity = 60`, and iterates through the sorted weights to calculate the total weight without exceeding the capacity. On the right, the 'Output' panel shows the result: 'Maximum weight loaded = 60' and '=== Code Execution Successful ==='. A 'Run' button is visible above the code editor.

```
main.c Run Output
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int compareDescending(const void *a, const void *b) {
5     return (*(int *)b - *(int *)a);
6 }
7
8 int main() {
9     int weights[] = {10, 20, 30, 40, 50};
10    int n = 5;
11    int max_capacity = 60;
12
13    int totalWeight = 0;
14
15    // Sort weights in descending order
16    qsort(weights, n, sizeof(int), compareDescending);
17
18    // Greedy selection
19    for (int i = 0; i < n; i++) {
```

Maximum weight loaded = 60

=== Code Execution Successful ===



The screenshot shows the Programiz C Online Compiler interface. On the left, the code editor displays a C program named 'main.c'. The code implements a greedy algorithm for the knapsack problem. It starts with an array of weights [5, 10, 15, 20, 25, 30, 35] and a maximum capacity of 60. The weights are sorted in descending order, resulting in [35, 30, 25, 20, 15, 10, 5]. The algorithm then iterates through the sorted weights, adding each weight to the total if it does not exceed the remaining capacity. The output shows that the maximum weight loaded is 60. On the right, the 'Output' panel displays the result: 'Maximum weight loaded = 60' and '=== Code Execution Successful ==='.

```
8 int main() {
9     int n = 7;
10    int max_capacity = 60;
11
12    int totalWeight = 0;
13
14    // Sort weights in descending order
15    qsort(weights, n, sizeof(int), compareDescending);
16
17    // Greedy selection
18    for (int i = 0; i < n; i++) {
19        if (totalWeight + weights[i] <= max_capacity) {
20            totalWeight += weights[i];
21        }
22    }
23
24    printf("Maximum weight loaded = %d\n", totalWeight);
25
26    return 0;
27 }
28 }
```

Output

Maximum weight loaded = 60

=== Code Execution Successful ===

## Time Complexity

Sorting the weights takes:

$O(n \log n)$

Traversing the array takes:

$O(n)$

Therefore:

Time Complexity =  $O(n \log n)$

## Space Complexity

The algorithm uses only a few extra variables apart from the input array.

Space Complexity =  $O(1)$  auxiliary space.

## Result

The heaviest item, 50, is selected first. Since adding any other item would exceed the capacity of 60, the maximum weight loaded .

**10. Given a list of item weights and a maximum capacity for each container, determine the minimum number of containers required to load all items using a greedy approach. The greedy approach should prioritize loading items into the current container until it is full before moving to the next container.**

**Test Case 1:**

**Input:**

**n = 7**

**weights = [5, 10, 15, 20, 25, 30, 35]**

**max\_capacity = 50**

**Output: 4**

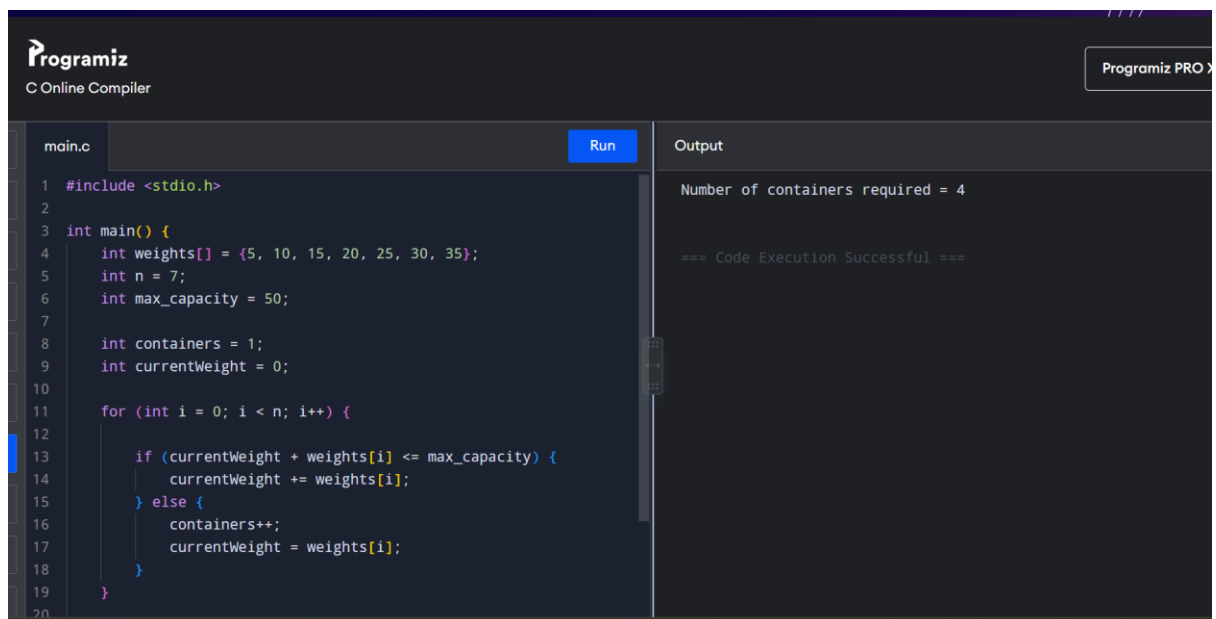
### Aim

To determine the minimum number of containers required to load all items using a greedy approach. Each container is filled as much as possible before moving to the next container.

### Algorithm

1. Read the item weights and maximum container capacity.
2. Start with the first container.
3. Add items to the current container as long as their total weight does not exceed the capacity.
4. If the next item cannot fit, open a new container.
5. Repeat until all items are loaded.
6. Count and display the total number of containers used.

### Screenshots of code



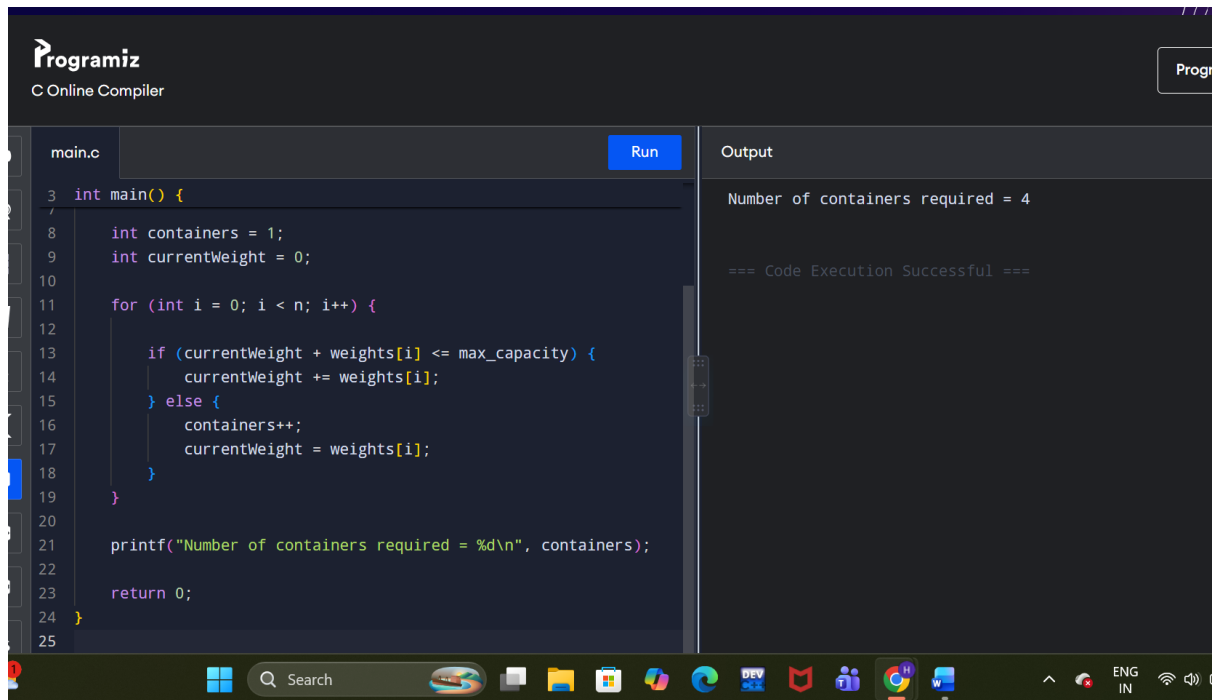
The screenshot displays the Programiz C Online Compiler interface. On the left, the code editor shows a C program named 'main.c' that implements a greedy algorithm for container loading. The code defines an array of weights, a maximum capacity of 50, and iterates through the items, adding them to the current container until it overflows, then starting a new container. On the right, the 'Output' panel shows the result: 'Number of containers required = 4' and '=== Code Execution Successful ==='.

```
1 #include <stdio.h>
2
3 int main() {
4     int weights[] = {5, 10, 15, 20, 25, 30, 35};
5     int n = 7;
6     int max_capacity = 50;
7
8     int containers = 1;
9     int currentWeight = 0;
10
11     for (int i = 0; i < n; i++) {
12
13         if (currentWeight + weights[i] <= max_capacity) {
14             currentWeight += weights[i];
15         } else {
16             containers++;
17             currentWeight = weights[i];
18         }
19     }
20 }
```

Output

Number of containers required = 4

=== Code Execution Successful ===



```
main.c Run Output
3 int main() {
4
5     int containers = 1;
6     int currentWeight = 0;
7
8     for (int i = 0; i < n; i++) {
9
10        if (currentWeight + weights[i] <= max_capacity) {
11            currentWeight += weights[i];
12        } else {
13            containers++;
14            currentWeight = weights[i];
15        }
16    }
17
18    printf("Number of containers required = %d\n", containers);
19
20    return 0;
21 }
22
23
24
25
```

Number of containers required = 4

=== Code Execution Successful ===

## Time Complexity

Each item is visited once.

Time Complexity =  $O(n)$

## Space Complexity

Only a few variables are used.

Space Complexity =  $O(1)$

## Result

The greedy algorithm successfully loads all items into 4 containers, with each container being filled as much as possible before moving to the next.

**11. Given a graph represented by an edge list, implement Kruskal's Algorithm to find the Minimum Spanning Tree (MST) and its total weight.**

**Test Case 1:**

**Input:**

**$n = 4$**

**$m = 5$**

**edges = [ (0, 1, 10), (0, 2, 6), (0, 3, 5), (1, 3, 15), (2, 3, 4) ]**

**Output:**

**Edges in MST: [(2, 3, 4), (0, 3, 5), (0, 1, 10)]**

**Total weight of MST: 19**

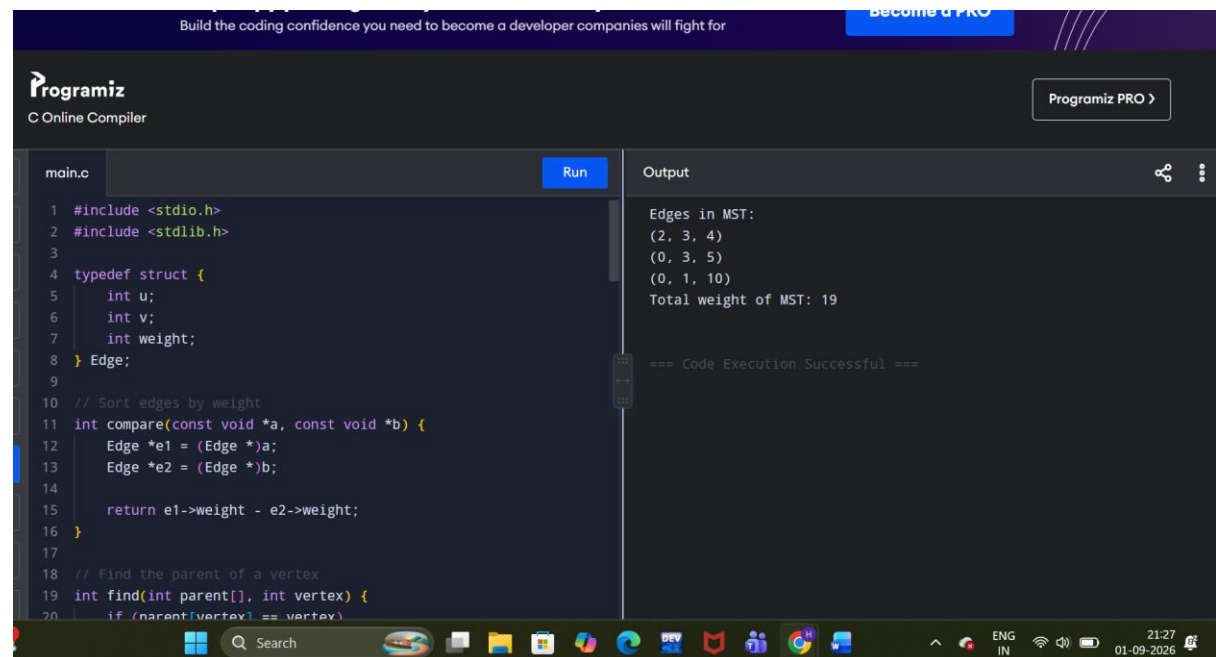
**Aim**

To implement Kruskal's Algorithm using an edge list to find the Minimum Spanning Tree (MST) of a connected graph and calculate its total weight.

### Algorithm

1. Read the number of vertices  $n$  and edges  $m$ .
2. Sort all edges in ascending order of weight.
3. Initially, consider every vertex as a separate set.
4. Pick the edge with the smallest weight.
5. Check whether adding the edge creates a cycle.
6. If it does not create a cycle, add it to the MST.
7. Continue until  $n - 1$  edges are selected.
8. Add the weights of the selected edges to obtain the total MST weight.

### Screenshots of code:



The screenshot displays the Programiz C Online Compiler interface. The code in `main.c` implements Kruskal's Algorithm. It defines an `Edge` struct with `u`, `v`, and `weight` fields. A `compare` function sorts edges by weight. A `find` function checks for cycles. The output shows the edges in the MST and the total weight.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct {
5     int u;
6     int v;
7     int weight;
8 } Edge;
9
10 // Sort edges by weight
11 int compare(const void *a, const void *b) {
12     Edge *e1 = (Edge *)a;
13     Edge *e2 = (Edge *)b;
14
15     return e1->weight - e2->weight;
16 }
17
18 // Find the parent of a vertex
19 int find(int parent[], int vertex) {
20     if (parent[vertex] == vertex)
```

Output:

```
Edges in MST:
(2, 3, 4)
(0, 3, 5)
(0, 1, 10)
Total weight of MST: 19

=== Code Execution Successful ===
```

```
main.c Run Output
26 // Union two sets
27 void unionSets(int parent[], int rank[], int u, int v) {
28     int rootU = find(parent, u);
29     int rootV = find(parent, v);
30
31     if (rootU != rootV) {
32         if (rank[rootU] < rank[rootV]) {
33             parent[rootU] = rootV;
34         }
35         else if (rank[rootU] > rank[rootV]) {
36             parent[rootV] = rootU;
37         }
38         else {
39             parent[rootV] = rootU;
40             rank[rootU]++;
41         }
42     }
43 }
44
45 void kruskal(Edge edges[], int n, int m) {
    ...
}
```

Edges in MST:  
(2, 3, 4)  
(0, 3, 5)  
(0, 1, 10)  
Total weight of MST: 19

=== Code Execution Successful ===

## Time Complexity

- Sorting edges:  $O(m \log m)$
- Union-Find operations: approximately  $O(m \alpha(n))$

Therefore:

Time Complexity =  $O(m \log m)$

## Space Complexity

- Parent array:  $O(n)$
- Rank array:  $O(n)$
- Edge list:  $O(m)$

Space Complexity =  $O(n + m)$

## Result

Kruskal's Algorithm successfully finds the Minimum Spanning Tree.

**12. Given a graph with weights and a potential Minimum Spanning Tree (MST), verify if the given MST is unique. If it is not unique, provide another possible MST.**

**Test Case 1:**

**Input:**

**n = 4**

**m = 5**

**edges = [ (0, 1, 10), (0, 2, 6), (0, 3, 5), (1, 3, 15), (2, 3, 4) ]**

**given\_mst = [(2, 3, 4), (0, 3, 5), (0, 1, 10)]**

**Output: Is the given MST unique? True**

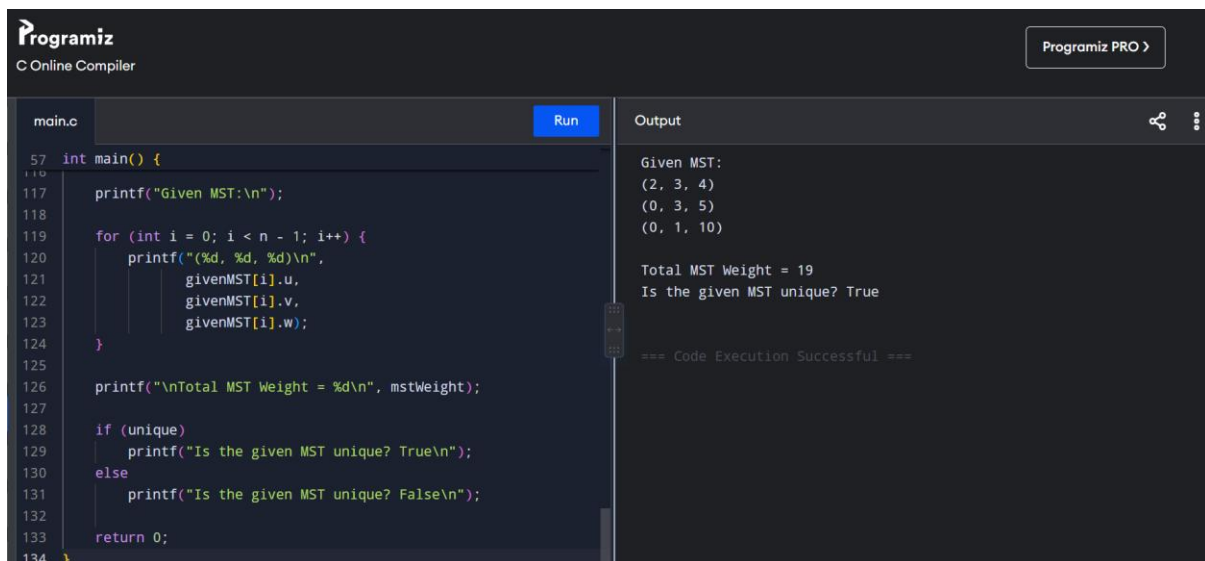
### Aim

To verify whether a given Minimum Spanning Tree (MST) is unique. If another MST with the same minimum total weight exists, the MST is not unique.

### Algorithm

1. Calculate the total weight of the given MST.
2. Check whether the given MST contains exactly  $n - 1$  edges.
3. For each edge in the given MST, temporarily remove it.
4. Find the minimum-weight edge that can reconnect the two resulting components.
5. If an alternative edge with the same weight can replace the removed edge and produce the same total MST weight, another MST exists.
6. If no such replacement exists for every MST edge, the given MST is unique.
7. Display the result.

### Screenshots of code:



The screenshot shows the Programiz C Online Compiler interface. The code in the editor is as follows:

```
main.c
116 int main() {
117     printf("Given MST:\n");
118
119     for (int i = 0; i < n - 1; i++) {
120         printf("(%d, %d, %d)\n",
121             givenMST[i].u,
122             givenMST[i].v,
123             givenMST[i].w);
124     }
125
126     printf("\nTotal MST Weight = %d\n", mstWeight);
127
128     if (unique)
129         printf("Is the given MST unique? True\n");
130     else
131         printf("Is the given MST unique? False\n");
132
133     return 0;
134 }
```

The output on the right is:

```
Output
Given MST:
(2, 3, 4)
(0, 3, 5)
(0, 1, 10)

Total MST Weight = 19
Is the given MST unique? True

*** Code Execution Successful ***
```

```
main.c Run Output

57 int main() {
89     for (int i = 0; i < n - 1; i++) {
99     }
100
101     if (removed == -1)
102         continue;
103
104     // Remove this edge and calculate another MST
105     int count;
106     int alternativeWeight =
107         calculateMST(edges, m, n, removed, -1, &count);
108
109     if (count == n - 1 &&
110         alternativeWeight == mstWeight) {
111
112         unique = 0;
113         break;
114     }
115 }
116

Given MST:
(2, 3, 4)
(0, 3, 5)
(0, 1, 10)

Total MST Weight = 19
Is the given MST unique? True

=== Code Execution Successful ===
```

## Time Complexity

For each MST edge, we perform an MST calculation over all edges.

Time Complexity =  $O(n \times m \alpha(n))$

For practical purposes, this is approximately:

$O(n \times m)$

## Space Complexity

The Union-Find parent array requires:

Space Complexity =  $O(n)$

## Result

The given MST is:

(2,3,4), (0,3,5), (0,1,10)

with total weight:

19

There is no alternative MST with the same total weight.



